

Mission E : Intrusion (IDS/IPS)

Contexte : BTS SIO 2ème année - Mise en place d'un système de détection d'intrusion sur une infrastructure virtualisée (VirtualBox).

Outil retenu : Snort 3 (N-IDS - Network Intrusion Detection System)

1. Théorie : Les IDPS (Intrusion Detection and Prevention Systems)

Dans le cadre de la sécurité des systèmes d'information, l'objectif est de garantir la **Confidentialité**, l'**Intégrité** et la **Disponibilité** (triade CIA). Face aux attaques (Virus, Spoofing, DoS), plusieurs systèmes de sécurité sont mis en œuvre :

- **Sécurité active (IPS)** : Filtrage et blocage des flux (ex: Firewall).
- **Sécurité passive (IDS)** : Détection et reconnaissance d'intrusions sans blocage du trafic.

Snort est un **N-IDS** (Network IDS). Il écoute le trafic réseau en temps réel via la bibliothèque `libpcap` (mode promiscuous) et effectue une recherche de motifs (*Pattern matching*) en comparant les paquets à une base de signatures.

Attention aux Faux Positifs / Faux Négatifs : Un faux positif est une alerte déclenchée à tort sur un trafic benign. Un faux négatif est une attaque réelle passée inaperçue. La configuration des règles doit limiter ces deux phénomènes.

2. Architecture de test sur VirtualBox

Pour réaliser des tests pertinents, le réseau suivant est configuré sur VirtualBox :

Machine	Rôle	Configuration Réseau
Machine 1 (Ubuntu Server 20.04)	Serveur IDS (Snort)	Carte 1 : NAT (Internet) Carte 2 : Réseau Interne (Écoute)

Machine 2 (Kali Linux / Windows)	Attaquant / Victime	1 carte : Réseau Interne (même réseau que la carte 2 de l'IDS)
---	---------------------	--

3. Procédure d'installation de Snort 3 (Ubuntu 20.04)

Snort 3 nécessite une compilation depuis les sources car il n'est pas dans les dépôts par défaut d'Ubuntu 20.04. Toutes les commandes sont exécutées en root (`sudo` ou `su -`).

Étape 1 : Pré-requis de compilation

```
apt update && apt upgrade -y
apt install -y build-essential autotools-dev libdumbnet-dev liblua5.2-dev
```

Étape 2 : Installation de la bibliothèque TPL (Template Processing Library)

```
cd /opt
wget https://github.com/TemplateLib/tpl/releases/download/v1.7/tpl-1.7.tar.gz
tar -xzf tpl-1.7.tar.gz
cd tpl-1.7
./configure && make && make install
```

Étape 3 : Installation de DAQ (Data Acquisition library)

```
cd /opt
wget https://github.com/snort3/libdaq/archive/refs/tags/v3.0.12.tar.gz
tar -xzf libdaq-3.0.12.tar.gz
cd libdaq-3.0.12
cmake -DCMAKE_INSTALL_PREFIX=/usr/local -DENABLE_PCAP=ON .
make && make install
```

Étape 4 : Compilation et installation de Snort 3

```
cd /opt
wget https://github.com/snort3/snort3/archive/refs/tags/v3.1.28.0.tar
tar -xzf snort3-3.1.28.0.tar.gz
cd snort3-3.1.28.0
configure --enable-sourcefire && make && make install
ldconfig
```

Étape 5 : Vérification de l'installation

```
snort -V
```

Cette commande doit retourner la version de Snort 3 installée sans erreur.

4. Configuration et Règles (Rules)

Snort fonctionne sur la base de règles définissant les signatures des attaques. La syntaxe générale d'une règle est :

```
action protocole @IP_src port_src -> @IP_dest port_dest
(options)
```

Téléchargement des règles communautaires

Suivant la procédure du TP, nous téléchargeons les règles officielles communautaires :

```
cd /tmp
wget https://www.snort.org/downloads/community/snort3-community-rules
tar -xvzf snort3-community-rules.tar.gz -C /usr/local/etc/rules
```

Il faut ensuite éditer le fichier de configuration principal `/usr/local/etc/snort/snort.lua` pour inclure ce fichier de règles en décommentant ou ajoutant la ligne :

```
include = "/usr/local/etc/rules/snort3-community.rules"
```

Création de règles personnalisées

Nous créons un fichier `/usr/local/etc/rules/local.rules` et y ajoutons 3 règles de différents types pour tester notre IDS :

Règle 1 : Détection d'un Ping (ICMP)

Déclenche une alerte si un ping est reçu (utile pour détecter des reconstructions de réseau).

```
alert icmp any any -> any any (msg:"ATTACK - ICMP Ping  
Detected"; sid:1000001; rev:1;)
```

Règle 2 : Détection d'un scan Nmap (SYN/FIN Scan)

Comme vu en cours, l'envoi de paquets avec les flags SYN et FIN simultanément est une anomalie caractéristique d'un scan furtif.

```
alert tcp any any -> any any (msg:"ATTACK - Nmap SYN/FIN  
Scan Detected"; flags:SF; sid:1000002; rev:1;)
```

Règle 3 : Détection d'une tentative d'accès à un fichier sensible via HTTP

Recherche d'une chaîne de caractères spécifique (*Pattern matching*) dans la charge utile (*payload*).

```
alert tcp any any -> any 80 (msg:"ATTACK - Attempt to access  
/etc/passwd via HTTP"; content:"GET /etc/passwd";  
sid:1000003; rev:1;)
```

N'oubliez pas d'ajouter `include` = `"/usr/local/etc/rules/local.rules"` dans le `snort.lua`.

5. Tests et Résultats

Pour lancer Snort en mode détection d'intrusion sur notre carte réseau interne (ex: `enp0s8`) :

```
snort -c /usr/local/etc/snort/snort.lua -i enp0s8 -A console
```

(L'option `-A console` permet d'afficher les alertes directement dans le terminal de manière lisible, au format IDMEF dans les logs standards).

Test 1 : Le Ping

Action : Depuis la Machine 2, on exécute `ping IP_MACHINE_1`.

Résultat attendu : Le terminal de Snort affiche instantanément l'alerte :

```
[**] [1:1000001:1] "ATTACK - ICMP Ping Detected" [**] [Priority: 0]
[ICMP] IP_MACHINE_2 -> IP_MACHINE_1
```

Test 2 : Le Scan de ports

Action : Depuis la Machine 2 (Kali), on lance `nmap -sF IP_MACHINE_1` (Scan FIN).

Résultat observé : Snort détecte les paquets avec le flag FIN seul. Si l'on fait un `nmap -sX IP_MACHINE_1` (Scan Xmas, qui met les flags FIN, URG, PSH), on peut tester la résilience. Pour forcer le test de notre règle : `nmap --scanflags SF IP_MACHINE_1`. L'alerte "SYN/FIN Scan Detected" s'affiche bien sur la console.

Test 3 : L'attaque Web

Action : Depuis la Machine 2, on utilise curl ou un navigateur pour envoyer : `curl http://IP_MACHINE_1/etc/passwd`.

Résultat observé : Snort analyse la couche 7 (HTTP), trouve la chaîne "GET /etc/passwd" et déclenche l'alerte correspondante avec le SID 1000003.

6. Conclusion

L'installation de Snort 3 en mode N-IDS permet de surveiller le trafic de manière passive. Nous avons mis en évidence la capacité de l'outil à effectuer de l'analyse protocolaire (ICMP, Flags TCP) et du pattern matching (recherche de chaîne HTTP). Bien que très puissant et modulaire (Snort 3 utilise Lua), la principale difficulté réside dans la rédaction précise des règles pour éviter une surcharge d'alertes (Faux Positifs) qui rendrait la lecture des logs impossible pour l'administrateur réseau.